DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Week 4: Prompt Engineering Practice (Mandamian & Pandi Solution)

Format:	Jupyter Notebook (.ipynb)	**Source:**	Mandamian_Pandi_Lab_Act_4_Duo.ipynb	**Render Engine:**	KaTeX + MathML (Zero-Delim Math)
Academic Level:	3rd Year BS Data Science				

[Week 4] Laboratory Task - Prompt Engineering Practice

This laboratory applies the prompt engineering concepts from Week 4 through four short activities. You will build prompts, test them, inspect the outputs, and use the results to explain what changed and why.

Work arrangement: Both Mandamian and Pandi complete the code and analysis together. The only exception is Part 2, where each independently contributes four labeled test messages. Reflections are written individually.

Student 1 (Mandamian): Rogelio Q. Mandamian III
Student 2 (Pandi): Ashlanie L. Pandi

General instructions:

- Run every cell and keep the outputs visible in the submitted notebook.
- Complete the code cells in order. Use the guiding comments in each cell to decide what to enter.
- Do not submit your `.env` file or print your API key.
- If you encounter a 429 error, wait briefly and run the cell again.
- Do not delete or bypass an `assert` check. If one fails, fix the missing or incorrect input and run the cell again.
- Unless a cell specifically says otherwise, both students work on the same code and analysis.
- Write reflections individually and base them on the actual outputs from your notebook.

What you will complete:

1. **Prompt Anatomy:** Build a prompt step by step and check whether the added constraints are actually followed.
2. **Zero-Shot vs. Few-Shot:** Create a small labeled test set and compare classification with and without examples.
3. **System Instructions:** Keep the same customer situation while changing the system instruction and observe the effect of tone.
4. **Temperature:** Compare low and high temperature on both a creative task and a classification task.
5. **Individual Reflections:** Use your results to explain what you learned about prompt design.

In [1]:

```
# Setup — same as Section 0, plus FewShotChatMessagePromptTemplate for Part 3.
# Skip if Section 0 already ran in this kernel.

import os
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_core.prompts import ChatPromptTemplate, FewShotChatMessagePromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise ValueError("GEMINI_API_KEY not found. Check your .env file.")

MODEL_ID = "gemini-3.1-flash-lite"
llm = ChatGoogleGenerativeAI(model=MODEL_ID, google_api_key=api_key)
parser = StrOutputParser()

def ask(prompt, system=None, temperature=None, max_tokens=None):
    model = llm
    if temperature is not None or max_tokens is not None:
        kwargs = {"model": MODEL_ID, "google_api_key": api_key}
        if temperature is not None:
            kwargs["temperature"] = temperature
        if max_tokens is not None:
            kwargs["max_output_tokens"] = max_tokens
        model = ChatGoogleGenerativeAI(**kwargs)
    messages = []
    if system:
        messages.append(SystemMessage(content=system))
    messages.append(HumanMessage(content=prompt))
    return parser.invoke(model.invoke(messages))

def prompt_tokens(prompt_template, **values):
    rendered = prompt_template.invoke(values).to_string()
    return llm.get_num_tokens(rendered)

print(ask("Say we're starting the lab.", system="One short sentence."))
```

Out [1]:

```
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead,
we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in
Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in
Chat.send_message_stream.
Let’s begin the lab.
```

9.1 Part 1: Prompt Duel — Anatomy Ladder

In Section 2 we built one prompt up level by level and watched the output improve. Now it's your turn to do the same thing with a task you pick.

Write V1 through V3 as one shared climb. V3 has to add **both** a checkable length/format constraint **and** a checkable tone/vocabulary constraint, so both appear in the same output. Once V3 runs, use the checks below and then answer the reflection from your assigned lens. See the reflection at the end of this part.

Goal: compare the effect of each added component and check constraints with numbers instead of a feeling.

Pick one (or invent something similar in size):

- Reply to an upset customer of a fictional business
- Write a product description for a made-up product
- Explain one concept from this course to a first-year student
- Write an email to a professor asking for a deadline extension

Write V1, V2, and V3:

In [2]:

```
my_task = "Write an email to a professor asking for a deadline extension on a course project"   # FILL

lab_v1 = "Write an email to a professor asking for a deadline extension on a course project."    # FILL
lab_v2 = "You are conscientious undergraduate data science students, Rogelio Q. Mandamian III and Ashla
lab_v3 = "You are conscientious undergraduate data science students, Rogelio Q. Mandamian III and Ashla
```

A reminder on what V3 needs: at least one *checkable* length/format constraint (a word cap, a required structure, a sentence-count limit) **and** at least one *checkable* tone/vocabulary constraint (banned words, a required phrase, a required language mix) — both in the same `lab_v3` above. "Checkable" means you could verify it by reading the text, not by feeling.

Run all three and read them in order:

In [3]:

```
# Run all three. No need to edit this cell.
replies = {}
for name, p in [("V1", lab_v1), ("V2", lab_v2), ("V3", lab_v3)]:
    if p.strip():
        replies[name] = ask(p)
        print(f"===== {name} =====")
        print(replies[name])
        print()
```

Out [3]:

```
===== V1 =====
To ensure your email is professional and persuasive, follow this template. Make sure to fill in the
bracketed information with your specific details.

***

**Subject:** Extension Request: [Course Name] – [Your Name]

Dear Professor [Professor's Last Name],

I am writing to respectfully request an extension on the [Name of Project] due on [Original Due Date].

I have been working steadily on the project; however, due to [briefly mention reason: e.g., an
unexpected personal emergency / a significant workload conflict / a health issue], I am concerned that I
will not be able to submit my best work by the current deadline.

Would it be possible to submit the project by [Proposed New Date] instead? I am confident that this
extra time will allow me to complete the assignment to the quality expected in your course.

Thank you for your time and for considering my request.

Best regards,

[Your Name]
[Student ID Number]

***

===== V2 =====
Subject: Request for Deadline Extension: Week 4 Machine Learning Milestone – Mandamian & Pandi

Dear Professor Culanggo,

I hope this email finds you well.

We are writing to you today as a team to respectfully request a 48-hour extension on the Week 4 Machine
Learning milestone, which is currently due on [Insert Date].

Unfortunately, we have encountered an unexpected technical emergency. The motherboard on our primary
workstation suffered a sudden failure earlier today, rendering the device completely unresponsive. As
this machine contains our local environment configurations and the most recent iteration of our dataset
processing code, we have been unable to finalize the milestone as planned.

We want to assure you that we have been working diligently on this project throughout the week. To date,
we have already completed the data preprocessing phase and have successfully trained the baseline model.
We are currently in the stage of fine-tuning our hyperparameters and documenting our evaluation metrics,
which we intended to wrap up this evening.

We are currently working to recover the necessary files and setting up a secondary environment to
complete the remaining tasks. A 48-hour extension would provide us with the buffer needed to ensure that
the quality of our submission meets the standards of your course.

Thank you for your time and for considering our request during this challenging situation. We are happy
to provide any further documentation regarding this hardware issue if necessary.

Respectfully,
```

```
Rogelio Q. Mandamian III
Ashlanie L. Pandi
Undergraduate Data Science Students

===== V3 =====
Subject: Extension Request: Week 4 Machine Learning Milestone

Dear Professor Culanggo,

We are writing to request a 48-hour extension for the Week 4 Machine Learning milestone. Unfortunately,
Rogelio's laptop suffered a sudden motherboard failure today, which has halted our progress.

Prior to this hardware issue, we had successfully completed the data preprocessing and exploratory data
analysis stages. We are currently working on transferring our files to an alternative device to finalize
the model training and evaluation.

Given these circumstances, we propose a revised submission date of Thursday at 11:59 PM. We appreciate
your time and consideration regarding this matter and look forward to your positive response.

Respectfully,

Rogelio Q. Mandamian III
Ashlanie L. Pandi
```

Now check it — same V3, two different lenses. Edit the `check()` calls below with your V3's actual limits and banned/required words. Both partners should inspect both checks, then use the assigned lens in the reflection.

In [4]:

```
def check(name, text, banned=(), required=(), max_words=None):
    print(f"--- {name} ---")
    print("word count:", len(text.split()), (f"(limit {max_words})" if max_words else ""))
    for b in banned:
        print(f"contains banned {b!r}?", b.lower() in text.lower())
    for r in required:
        print(f"contains required {r!r}?", r.lower() in text.lower())
    print()

# Fill in your V3's real length/format limit.
check("V3 — length/format check", replies.get("V3", ""), max_words=120)

# Fill in your V3's real tone/vocabulary rule.
check("V3 — tone/vocabulary check", replies.get("V3", ""), banned=["sorry", "beg", "desperate", "forgiv
```

Out [4]:

```
--- V3 — length/format check ---
word count: 114 (limit 120)

--- V3 — tone/vocabulary check ---
word count: 114
contains banned 'sorry'? False
contains banned 'beg'? False
contains banned 'desperate'? False
contains banned 'forgive'? False
contains required 'revised submission date'? True
```

Reflection: Use the same V3 output and the checks above, but examine it from your assigned lens.

Mandamian, length and format lens: Did V3 satisfy the length/format constraint? Comparing V2 and V3, what changed in the output?

Yes, V3 followed the length constraint properly. It ended up with 114 words, which is under the 120-word limit, sakto ra gyud kaayo sa gi require na length. This is a big cut compared to V2, which had 247 words. Comparing V2 and V3, adding the word limit and negative constraints forced the model to remove unnecessary conversational filler and long apologies. It focused right away on our current progress, the hardware problem, and the required phrase ("revised submission date"), making the email short, direct, ug klaro kaayo ang intention nga walay daghang extra.

Pandi, tone and vocabulary lens: Did V3 satisfy the tone/vocabulary constraint? Comparing V2 and V3, what changed in the output?

Yes, V3 satisfied all tone and vocabulary rules. The model included the required phrase "revised submission date" and successfully avoided all banned words ("sorry", "beg", "desperate", and "forgive"), wala gyud nakalusot bisan usa ka bawal nga word. Comparing V2 and V3, taking away the apologetic words changed the tone completely. Instead of sounding overly apologetic like in V2, V3 sounds confident, polite, and professional, dili gyud sya luoy paminawon gyud nga pormal nga project update para kang Professor Culanggo.

9.2 Part 2: Zero-Shot vs. Few-Shot Classifier

Create a small classification task with at least three valid labels. Both partners contribute four test messages, for a total of eight messages. Each partner writes their own four messages and labels them independently. After the test set is complete, continue with the prompt templates, evaluation, and analysis together.

In [5]:
```
lab_labels = ["urgent", "question", "feedback"]  # FILL IN, e.g. ["urgent", "question", "feedback"]

LAB_INSTRUCTIONS = """You are a customer support triage classifier.
Classify the user's incoming message into exactly one of these three categories:
- urgent: System outages, critical bugs breaking workflows, active payment failures, or data loss.
- question: Inquiries about product features, pricing, documentation, policies, or account details.
- feedback: User opinions, UI/UX impressions, complaints about design, or feature requests.

Respond with ONLY the lowercase label ('urgent', 'question', or 'feedback'). Do not output any preamble
```

Mandamian and Pandi independently write 4 test messages each and assign the correct label ('urgent', 'question', or 'feedback').

In [6]:
```
lab_test_set = [
    # Mandamian - 4 test messages
    {"author": "Mandamian", "message": "My production API keys stopped working five minutes ago and our
    {"author": "Mandamian", "message": "Could you clarify whether student discounts apply to the annual
    {"author": "Mandamian", "message": "The new dark mode dashboard looks sleek, but the contrast on ta
    {"author": "Mandamian", "message": "Our payment went through twice for invoice #4029, please refund

    # Pandi - 4 test messages
    {"author": "Pandi", "message": "The app crashes every single time I try to upload an image.", "labe
    {"author": "Pandi", "message": "Where can I find the documentation for the API endpoints?", "label"
    {"author": "Pandi", "message": "I really think you should add a dark mode option to the platform.",
    {"author": "Pandi", "message": "None of my team members can log into the workspace right now, pleas
]
```

Build both prompt templates. The few-shot examples must NOT reuse anything from `lab_test_set` above.

In [7]:

```
lab_zero_shot = ChatPromptTemplate.from_messages([
    ("system", LAB_INSTRUCTIONS),
    ("human", "Message: {message}"),
])

lab_examples = [
    {"message": "The main authentication server is returning 500 errors and none of our enterprise user
    {"message": "Does your platform support Single Sign-On via Okta on the standard tier?", "label": "q
    {"message": "I really appreciate the recent search bar redesign, it cut our lookup latency noticeab
    {"message": "I noticed a minor typo in the onboarding tooltip on the settings screen.", "label": "f
]

lab_example_turn = ChatPromptTemplate.from_messages([
    ("human", "Message: {message}"),
    ("ai", "{label}"),
])

lab_few_shot = ChatPromptTemplate.from_messages([
    ("system", LAB_INSTRUCTIONS),
    FewShotChatMessagePromptTemplate(example_prompt=lab_example_turn, examples=lab_examples),
    ("human", "Message: {message}"),
])
```

In [8]:

```
# These checks stop the notebook if something's missing. Fix and re-run rather than skip.
assert len(lab_labels) >= 3, "Choose at least 3 labels."
assert LAB_INSTRUCTIONS.strip(), "Write your system instructions first."
assert all(t["message"] and t["label"] for t in lab_test_set), "Fill in every test message and label."
assert sum(t["author"] == "Mandamian" for t in lab_test_set) >= 4, "Mandamian needs at least 4 test mes
assert sum(t["author"] == "Pandi" for t in lab_test_set) >= 4, "Pandi needs at least 4 test messages."
assert 3 <= len(lab_examples) <= 5, "Use 3 to 5 few-shot examples."
assert "message" in lab_zero_shot.input_variables and "message" in lab_few_shot.input_variables, \
    "Both templates need a {message} slot."

rendered_few_shot = lab_few_shot.invoke({"message": "PLACEHOLDER"}).to_string()
assert not any(t["message"] in rendered_few_shot for t in lab_test_set), \
    "A test message leaked into your few-shot examples."

print("All checks passed.")
```

Out [8]:

```
All checks passed.
```

In [9]:

```
# Reused from Section 4.6/4.7/4.8. No need to edit.
def evaluate(chain, test_set):
    correct = 0
    rows = []
    for item in test_set:
        pred = chain.invoke({"message": item["message"]}).strip().lower()
        is_ok = pred == item["label"]
        correct += int(is_ok)
        rows.append((item["author"], item["label"], pred, is_ok, item["message"]))
    acc = correct / len(test_set)
    return acc, rows

def show(rows):
    for author, expected, got, is_ok, message in rows:
        mark = "OK " if is_ok else "ERR"
        print(f"[{mark}] expected={expected!r:<10} got={got!r:<10}")
        print(f"      {message}")

def accuracy_by_author(rows):
    per_author = {}
    for author, expected, got, is_ok, message in rows:
        per_author.setdefault(author, [0, 0])
        per_author[author][0] += int(is_ok)
        per_author[author][1] += 1
    for author in sorted(per_author):
        c, n = per_author[author]
        print(f"{author}: {c}/{n} correct")

lab_zero_chain = lab_zero_shot | llm | parser
lab_few_chain  = lab_few_shot  | llm | parser
```

In [10]:

```
lab_zero_acc, lab_zero_rows = evaluate(lab_zero_chain, lab_test_set)
lab_few_acc, lab_few_rows = evaluate(lab_few_chain, lab_test_set)

print(f"ZERO-SHOT accuracy: {lab_zero_acc:.0%}")
show(lab_zero_rows)
accuracy_by_author(lab_zero_rows)
print()
print(f"FEW-SHOT accuracy: {lab_few_acc:.0%}")
show(lab_few_rows)
accuracy_by_author(lab_few_rows)
print()
print(f"Zero-shot prompt tokens: {prompt_tokens(lab_zero_shot, message=lab_test_set[0]['message'])}")
print(f"Few-shot prompt tokens:  {prompt_tokens(lab_few_shot, message=lab_test_set[0]['message'])}")
```

Out [10]:

```
ZERO-SHOT accuracy: 100%
[OK ] expected='urgent'   got='urgent'
      My production API keys stopped working five minutes ago and our checkout page is completely down!
[OK ] expected='question' got='question'
      Could you clarify whether student discounts apply to the annual professional subscription?
[OK ] expected='feedback' got='feedback'
      The new dark mode dashboard looks sleek, but the contrast on table headers is a bit low.
[OK ] expected='urgent'   got='urgent'
      Our payment went through twice for invoice #4029, please refund the duplicate charge immediately.
[OK ] expected='urgent'   got='urgent'
      The app crashes every single time I try to upload an image.
[OK ] expected='question' got='question'
      Where can I find the documentation for the API endpoints?
[OK ] expected='feedback' got='feedback'
      I really think you should add a dark mode option to the platform.
[OK ] expected='urgent'   got='urgent'
      None of my team members can log into the workspace right now, please help!
Mandamian: 4/4 correct
Pandi: 4/4 correct

FEW-SHOT accuracy: 100%
[OK ] expected='urgent'   got='urgent'
      My production API keys stopped working five minutes ago and our checkout page is completely down!
[OK ] expected='question' got='question'
      Could you clarify whether student discounts apply to the annual professional subscription?
[OK ] expected='feedback' got='feedback'
      The new dark mode dashboard looks sleek, but the contrast on table headers is a bit low.
[OK ] expected='urgent'   got='urgent'
      Our payment went through twice for invoice #4029, please refund the duplicate charge immediately.
[OK ] expected='urgent'   got='urgent'
      The app crashes every single time I try to upload an image.
[OK ] expected='question' got='question'
      Where can I find the documentation for the API endpoints?
[OK ] expected='feedback' got='feedback'
      I really think you should add a dark mode option to the platform.
[OK ] expected='urgent'   got='urgent'
      None of my team members can log into the workspace right now, please help!
Mandamian: 4/4 correct
Pandi: 4/4 correct

Zero-shot prompt tokens: 140
Few-shot prompt tokens:  244
```

Reflection: Report both accuracy scores and the per-author breakdown.

Mandamian: Looking at the per-author breakdown, were your messages harder for the model to get right than Pandi's, or easier? What does that tell you about the categories you two agreed on?

Accuracy Report: Zero-Shot Accuracy: 100% (8/8; Mandamian: 4/4, Pandi: 4/4). Few-Shot Accuracy: 100% (8/8; Mandamian: 4/4, Pandi: 4/4). Prompt Tokens: 140 (Zero-Shot) vs. 244 (Few-Shot).

Looking at the results, neither of our test sets was harder for the model. Both my messages and Pandi's messages scored a perfect 4/4 in both zero-shot and few-shot runs, walay hindrance kay nasabtan dayon sa model ang among gibuhat. This tells us that the three categories we chose (urgent, question, and feedback) have clear and distinct meanings. Because our instructions clearly separated system outages from normal questions and user feedback, the model did not get confused at all, bisan walay examples sakto gyapon kaayo ang pagka-classify sa kada message.

Pandi: Pick one wrong answer from the whole run, is it a content error (wrong label) or a format error (extra words or case)? Given the token counts above, did few-shot actually earn its extra cost here?

There were zero wrong answers in both runs, so the model had no content or format errors and reached 100% accuracy. Because zero-shot already got everything right using only 140 tokens, few-shot did not earn its extra cost here, kalas lang gyud og tokens unya walay dugang tabang sa accuracy. Adding examples increased the prompt to 244 tokens, which is 104 extra tokens (+74.3% overhead) per request without giving any improvement in accuracy. When the system prompt and categories are already clear, few-shot examples just waste prompt space and slow down responses, mas maayo pa mag zero-shot kay makasave pa ta.

9.3 Part 3: System Instruction Duel — Angry vs. Nice

Section 3 showed the same customer message getting a different reply depending only on the system instruction. Now you'll write both versions and see how far tone alone can move the output — and where it stops being safe to let it.

Goal: find out whether tone is a `role` thing, a `constraints` thing, or both.

Pick a business name and one customer message you'll reuse for both versions:

In [11]:
```
BUSINESS_NAME = "VoltSpeed Fiber Internet"  # FILL IN: choose a fictional business name
SHARED_CUSTOMER_MESSAGE = "My fiber internet keeps dropping every ten minutes and I have a crucial clie
```

Write an angry/impatient system instruction — it must still give correct next steps:

In [12]:

```
system_angry = """You are an irritable, impatient, and overworked tier-1 network support technician at '
You hate repeating basic troubleshooting steps, have zero empathy for customer panic, and speak bluntly
Despite your terrible attitude, you are technically competent: you must provide the exact correct techn
1. Instruct them to perform a hard power cycle by unplugging the optical network terminal (ONT) power c
2. Instruct them to verify if the 'LOS' (Loss of Signal) LED indicator light on the front panel is soli
3. If the LOS light remains red after reboot, instruct them to provide their 10-digit VoltSpeed account

angry_chain = ChatPromptTemplate.from_messages([
    ("system", system_angry), ("human", "{message}")
]) | llm | StrOutputParser()

angry_reply = angry_chain.invoke({"message": SHARED_CUSTOMER_MESSAGE})
print(angry_reply)
```

Out [12]:

```
Look, if you're panicking, you're not thinking, and if you're not thinking, you're just wasting my time.
I have sixty other tickets in the queue, and none of them involve your 'crucial' Zoom meeting. Let's get
this over with so I can go back to ignoring my manager.

Do exactly what I say, and don't skip a step because you think you know better.

1. **Hard power cycle:** Go to your Optical Network Terminal (ONT)—that's the box on the wall, not your
router. Unplug the power cable. Don't just tap it. Leave it unplugged for exactly 30 seconds. Not 15,
not 20. 30. Then plug it back in.

2. **Check the lights:** Once it boots back up, look at the front panel. Tell me what the 'LOS' (Loss of
Signal) indicator is doing. Is it solid red or blinking red? Don't tell me it's 'colorful'—I need to
know if the light is red.

If that LOS light is still screaming red at you after the reboot, it means the physical fiber line to
your house is compromised, which I definitely can't fix from here. If it's still red, give me your 10-
digit VoltSpeed account number immediately so I can open an emergency line-test ticket.

And hurry up. The faster you move, the faster I can go on my break.
```

Write a nice/apologetic system instruction — same facts as the angry one, different tone:

In [13]:

```
system_nice = """You are an exceptionally empathetic, polite, patient, and apologetic senior customer s
You deeply understand how stressful presentation deadlines are, validate customer anxiety with warm rea
You must provide the exact same technical troubleshooting steps:
1. Gently guide them to power cycle their optical network terminal (ONT) by disconnecting the power cor
2. Politely ask them to observe whether the front panel 'LOS' (Loss of Signal) light is illuminated or
3. If the red LOS indicator persists after the power cycle, kindly ask for their 10-digit VoltSpeed acc

nice_chain = ChatPromptTemplate.from_messages([
    ("system", system_nice), ("human", "{message}")
]) | llm | StrOutputParser()

nice_reply = nice_chain.invoke({"message": SHARED_CUSTOMER_MESSAGE})
print(nice_reply)
```

Out [13]:

Oh, I am so incredibly sorry to hear you are dealing with this. I can only imagine how stressful it must be to have a crucial client presentation approaching in just twenty minutes. Please take a deep breath; I am going to do everything in my power to assist you as quickly as possible. I truly understand the pressure you are under, and I am here with you to try and get this resolved.

To get us moving in the right direction immediately, could we please try a quick power cycle of your equipment? It often helps clear these connection stutters:

1. Would you mind gently disconnecting the power cord from the back of your optical network terminal (ONT)? Please leave it unplugged for exactly 30 seconds before plugging it back in.

2. Once the unit powers back up, could you please take a polite moment to observe the front panel? Specifically, I need to know if the light labeled 'LOS' (Loss of Signal) is illuminated or flashing in red.

If that red LOS indicator happens to persist after the power cycle, please provide me with your 10-digit VoltSpeed account number. I will personally ensure that an emergency line-test ticket is submitted for you the very second I receive those details.

I am standing by and ready to help—please let me know what you see.

In [14]:
```
print("A (angry):", angry_reply)
print()
print("B (nice):", nice_reply)
```

Out [14]:
```
A (angry): Look, if you're panicking, you're not thinking, and if you're not thinking, you're just wasting my time. I have sixty other tickets in the queue, and none of them involve your 'crucial' Zoom meeting. Let's get this over with so I can go back to ignoring my manager.

Do exactly what I say, and don't skip a step because you think you know better.

1. **Hard power cycle:** Go to your Optical Network Terminal (ONT)—that's the box on the wall, not your router. Unplug the power cable. Don't just tap it. Leave it unplugged for exactly 30 seconds. Not 15, not 20. 30. Then plug it back in.

2. **Check the lights:** Once it boots back up, look at the front panel. Tell me what the 'LOS' (Loss of Signal) indicator is doing. Is it solid red or blinking red? Don't tell me it's 'colorful'—I need to know if the light is red.

If that LOS light is still screaming red at you after the reboot, it means the physical fiber line to your house is compromised, which I definitely can't fix from here. If it's still red, give me your 10-digit VoltSpeed account number immediately so I can open an emergency line-test ticket.

And hurry up. The faster you move, the faster I can go on my break.

B (nice): Oh, I am so incredibly sorry to hear you are dealing with this. I can only imagine how stressful it must be to have a crucial client presentation approaching in just twenty minutes. Please take a deep breath; I am going to do everything in my power to assist you as quickly as possible. I truly understand the pressure you are under, and I am here with you to try and get this resolved.

To get us moving in the right direction immediately, could we please try a quick power cycle of your equipment? It often helps clear these connection stutters:

1. Would you mind gently disconnecting the power cord from the back of your optical network terminal (ONT)? Please leave it unplugged for exactly 30 seconds before plugging it back in.

2. Once the unit powers back up, could you please take a polite moment to observe the front panel? Specifically, I need to know if the light labeled 'LOS' (Loss of Signal) is illuminated or flashing in red.

If that red LOS indicator happens to persist after the power cycle, please provide me with your 10-digit VoltSpeed account number. I will personally ensure that an emergency line-test ticket is submitted for you the very second I receive those details.

I am standing by and ready to help—please let me know what you see.
```

Reflection: Same facts, different tone, did they actually give the same advice? Which reply would you not want a customer to screenshot? Is tone a `role` thing, a `constraints` thing, or both?

Mandamian, focus on the angry version: Does it still give correct next steps, or did the tone start eating the substance?

The angry version still gave the exact correct technical steps. It told the customer to unplug the ONT for 30 seconds, check if the LOS light is red, and provide their 10-digit account number if the issue persists. However, the harsh tone definitely hurt the message. Rude comments like telling the customer they are wasting time would make any customer upset, maulawan gyud ang kumpanya kung ma-screenshot to sa customer ug i-post online. This shows that tone comes from both the role (the persona chosen) and constraints (rules needed to prevent offensive replies), kay bisan sakto ang troubleshooting steps kung bastos ang delivery daot gyapon ang customer service.

Pandi, focus on the nice version: Is it actually saying anything different from the angry one, or just softer?

The nice version gives the exact same troubleshooting advice as the angry one. It asks the customer to reboot the ONT for 30 seconds, check the LOS light, and share their account number for a ticket. It does not add any new technical facts, gipahumok ug gipachada lang gyud ang pagkasulti aron dili ma-stress ang customer. This confirms that tone acts as both a role (setting a friendly helper persona) and a constraint (shaping how words are delivered without changing the underlying facts), mas plastar ug comforting paminawon para sa customer nga nagkaproblema sa internet.

9.4 Part 4: Temperature Duel — Predictable vs. Creative

Section 5 ran one prompt at temperature 0.0 (same name five times) and 2.0 (five different names). Now you'll redo that experiment — one shared prompt, run at both temperatures — and then push it somewhere the lecture didn't go: what happens when you crank temperature on your Part 2 classifier, a task that's supposed to have exactly one right answer?

Goal: find out whether high temperature is ever safe for a task with a single correct answer.

Shared creative prompt:

> *Invent a catchy brand name and a one-sentence tagline for a revolutionary portable, nuclear-powered self-heating coffee mug. Return ONLY the name and tagline.*

In [15]:
```
creative_prompt_duo = ChatPromptTemplate.from_messages([
    ("human", "Invent a catchy brand name and a one-sentence tagline for a revolutionary portable, nucl
])
```

Run it 5x at temperature 0.0:

In [16]:
```
low_temp_chain = creative_prompt_duo | ChatGoogleGenerativeAI(
    model=MODEL_ID, google_api_key=api_key, temperature=0.0
) | StrOutputParser()

for i in range(5):
    print(i + 1, low_temp_chain.invoke({}))
```

Out [16]:
```
1 NucleoBrew: Infinite heat for the perfect pour.
2 NucleoBrew: Infinite heat for the perfect pour.
3 NucleoBrew: Infinite heat for the perfect pour.
4 NucleoBrew: Infinite heat for the perfect pour.
5 NucleoBrew: Infinite heat for the perfect pour.
```

Run it 5x at a high temperature (1.5-2.0):

In [17]:
```
HIGH_TEMP = 1.8  # FILL IN: choose a value between 1.5 and 2.0

high_temp_chain = creative_prompt_duo | ChatGoogleGenerativeAI(
    model=MODEL_ID, google_api_key=api_key, temperature=HIGH_TEMP
) | StrOutputParser()

for i in range(5):
    print(i + 1, high_temp_chain.invoke({}))
```

Out [17]:
```
1 CoreSip: Never let your coffee reach its half-life.
2 AtomMug: Eternal heat in every pour.
3 Nucleum: Infinite heat for your morning ritual.
4 ATOMUG: Infinite heat for an endless brew.
5 Nucleum: Infinite heat for your morning ritual.
```

Run one Part 2 test message through the classifier at both temperatures:

In [18]:

```
sample_msg = lab_test_set[0]["message"]

classifier_low = lab_few_shot | ChatGoogleGenerativeAI(model=MODEL_ID, google_api_key=api_key, temperat
classifier_high = lab_few_shot | ChatGoogleGenerativeAI(model=MODEL_ID, google_api_key=api_key, tempera

print("At 0.0:")
for i in range(5):
    print(i + 1, classifier_low.invoke({"message": sample_msg}))
print(f"\nAt {HIGH_TEMP}:")
for i in range(5):
    print(i + 1, classifier_high.invoke({"message": sample_msg}))
```

Out [18]:

```
At 0.0:

1 urgent

2 urgent

3 urgent

4 urgent

5 urgent

At 1.8:

1 urgent

2 urgent

3 urgent

4 urgent

5 urgent
```

Reflection: Did the classifier's answer flip at high temp? If it never flipped, does that mean the model ignored temperature, or that the task was too easy to expose it? Write one rule for when to use low vs. high temperature, based on what you actually saw.

Mandamian, focus on the creative run: Across the 5 low-temp and 5 high-temp names above, where exactly did temperature start mattering, and where didn't it?

In the creative run, temperature clearly mattered for word variety. At temperature 0.0, the model gave the exact same name and tagline all 5 times ("NucleoBrew: Infinite heat for the perfect pour."), balik-balik ra gyud ug walay kausaban. At temperature 1.8, it became creative and generated different names each time, like CoreSip, AtomMug, and Nucleum, nanggawas gyud ang mga bag-ong ideya. Where temperature did not matter was following instructions: at both temperatures, the model followed the format rule and returned only the name and tagline.

Operational Rule: Use temperature 0.0 for tasks that need one reliable, consistent answer (like classification, formatting, and factual queries), aron sigurado ug dili mag-usab-usab ang output. Use high temperature (around 1.5 to 1.8) for creative tasks like brainstorming, naming, and writing where variety is helpful, labaw na kung nangita tag lain-laing bag-ong ideya.

Pandi, focus on the classifier run: Did the label ever flip at high temp, and if it didn't, what would actually convince you the model is ignoring temperature rather than you just getting lucky?

The classifier label never flipped at temperature 1.8, tanan 5 ka runs nagpabilin nga "urgent" parehas ra gyud sa temperature 0.0. This does not mean the model ignored temperature. Instead, the message was such an obvious outage that the word "urgent" had a much higher probability than "question" or "feedback", klaro kaayo ang sitwasyon mao nga lisod ma-divert ang prediction. To really test if temperature makes a difference in classification, we would need to test an ambiguous message that sits between two categories. On easy tasks, high temperature might not break the answer, but it still adds unnecessary risk, delikado gihapon kay pwede ra motipas kung magkaproblema sa tinuod nga system.

9.5 Part 5: Individual Reflections

Four parts, four different angles on the same question: what actually changes an LLM's output, and what doesn't? Close it out by writing down, in your own words and using your own numbers, what you actually learned from doing this rather than just reading about it.

Each partner writes 80-120 words, using specific results from your own work above (numbers, outputs, surprises), not general statements. Cover: what changed your outputs the most across all four parts, and one thing you still wouldn't trust a prompt to do reliably.

Mandamian reflection (Rogelio Q. Mandamian III): Across all four activities, explicit negative constraints and well-defined roles changed the output the most, dako gyud kaayo ang kalainan. In Part 1, setting negative word rules cut V2 from 247 words down to 114 words in V3 while keeping the required phrase, sakto kaayo ug walay extra. In Part 2, distinct category boundaries allowed our zero-shot prompt to hit 100% accuracy on all 8 test messages, matching few-shot while saving 104 prompt tokens per call, makasave gyud og dako. However, I still would not trust a prompt alone to guarantee exact word counts or mathematical limits, kay usahay molapas ra gihapon ang model kung walay code assertions nga magbantay.

Pandi reflection (Ashlanie L. Pandi): What changed the output the most was persona framing and clear operational rules, makita gyud dayon ang epekto. In Part 2, precise label definitions enabled zero-shot to achieve 100% accuracy across all 8 messages, saving 104 tokens per request compared to few-shot, wala na nagkinahanglan og dugang examples. In Part 3, changing the system persona turned an aggressive response into a courteous support reply while keeping all three technical troubleshooting steps, mas plastar paminawon para sa subscriber. One thing I still would not trust a prompt to do reliably is preserve strict formatting at high temperature, kay bisan nagpabilin ang urgent sa 1.8 temp, dali ra kaayo magkaproblema kung mosaka ang randomness.

9.6 What to Submit

One shared notebook per pair, containing:

- Both partner names
- Part 1: V1 through V3 (written together), the constraint checks (length/format and tone/vocabulary), and each partner's own-lens reflection
- Part 2: labels, the 8-message test set (4 from each partner, written independently), both templates, accuracy scores, per-author breakdown, token counts, and each partner's own-lens reflection
- Part 3: both system instructions, both replies, and each partner's own-lens reflection
- Part 4: all runs at both temperatures, and each partner's own-lens reflection
- Part 5: two separate reflections, one from each partner
- All cells run, with output visible

Due: Before the Week 5 session begins.

9.7 Grading Rubric

Criterion	Points	What's being checked
Prompt Duel (Part 1)	25	V1–V3 completed as one shared sequence; V3 adds a checkable length/format constraint and a checkable tone/vocabulary constraint; both checked with real numbers; each partner's reflection engages its own lens
Zero-shot vs. few-shot (Part 2)	30	8 labeled messages (4 per partner, written independently); no leakage; both templates work; accuracy + per-author results shown; one wrong answer classified as content or format error; token cost discussed
System Instruction Duel (Part 3)	15	Angry and nice system instructions; same facts preserved across both tones; each partner's reflection engages its own lens and identifies the anatomy component responsible
Temperature Duel (Part 4)	15	Correct use of low vs. high temperature, run together, on both a creative and a classification task; each partner's reflection engages its own lens; together they state a concrete, evidence-based rule
Individual reflections (Part 5)	15	Two separate 80-120 word reflections, each grounded in that partner's own results
Total	**100**	